Perfect. Your **Intel i5-8250 + 24 GB RAM + Qwen3:4B** setup is more than sufficient for **Phase 1**. In fact, the 4B model will be much faster during development than an 8B model.

Let's continue and make everything work together.

Step 5: Install Docker Desktop

Verify the installation:

```bash
docker --version
```

Expected output:

```
Docker version 28.x.x
```

Verify that Docker is running:

```bash
docker ps
```

If you don't receive an error, Docker is working.

Step 6: Pull and start FalkorDB

Pull the image:

```bash
docker pull falkordb/falkordb:latest
```

Start FalkorDB:

```bash
docker run -d ^
--name falkordb ^
-p 6379:6379 ^
falkordb/falkordb:latest
```

Verify that the container is running:

Bash

```
docker ps
```

You should see something similar to:

```
CONTAINER ID   IMAGE                    PORTS
abc123         falkordb/falkordb        0.0.0.0:6379->6379
```

Step 7: Create the project

Bash

```
mkdir agentic-text2sql

cd agentic-text2sql
```

Create a virtual environment:

Bash

```
python -m venv .venv
```

Activate it:

Command Prompt

Bash

```
.venv\Scripts\activate
```

PowerShell

Bash

```
.\.venv\Scripts\Activate.ps1
```

Verify:

Bash

```
python --version
```

Step 8: Create the folder structure

```
agentic-text2sql
│
├── app
│
├── agents
│
├── graph
│
├── database
│
├── frontend
│
├── data
│
├── tests
│
├── .env
│
├── requirements.txt
│
└── README.md
```

Create the folders:

```bash
mkdir app
mkdir agents
mkdir graph
mkdir database
mkdir frontend
mkdir data
mkdir tests
```

Create the files:

```bash
echo.>.env
echo.>requirements.txt
echo.>README.md
```

Step 9: Install dependencies

Install everything in one command:

Bash

```
pip install fastapi uvicorn sqlalchemy ollama falkordb python-dotenv pydantic
```

Save the dependencies:

Bash

```
pip freeze > requirements.txt
```

Verify:

Bash

```
pip list
```

Step 10: Download the Chinook database

Download the SQLite version from:

Chinook database releases

Rename:

```
Chinook_Sqlite.sqlite
```

to:

```
chinook.db
```

Move it to:

```
agentic-text2sql
└── data
    └── chinook.db
```

Verify:

Bash

```
dir data
```

Step 11: Test SQLite

Create:

```
database/test_sqlite.py
```

Add:

Python Run

```python
import sqlite3
from pathlib import Path

db_path = Path("../data/chinook.db")

connection = sqlite3.connect(db_path)

cursor = connection.cursor()

cursor.execute(
    """
    SELECT Name
    FROM Artist
    LIMIT 10
    """
)

rows = cursor.fetchall()

for row in rows:
    print(row[0])

connection.close()
```

Run:

Bash

```bash
cd database

python test_sqlite.py
```

Expected output:

```
AC/DC
Accept
Aerosmith
Alanis Morissette
Alice In Chains
```

```
Antônio Carlos Jobim
Apocalyptica
Audioslave
BackBeat
Billy Cobham
```

Step 12: Test FalkorDB

Return to the project root:

Bash

```
cd ..
```

Create:

```
graph/test_falkordb.py
```

Add:

Python Run

```
from falkordb import FalkorDB

db = FalkorDB(
    host="localhost",
    port=6379
)

graph = db.select_graph("text2sql")

graph.query(
    """
    CREATE (:Table {
        name:'Artist',
        description:'Artist table'
    })
    """
)

result = graph.query(
    """
    MATCH (n:Table)
    RETURN n.name
    """
)

for row in result.result_set:
```

```
    print(row)
```

Run:

```bash
python graph/test_falkordb.py
```

Expected output:

```
Artist
```

Step 13: Build a minimal FastAPI backend

Create:

```
app/main.py
```

Add:

```python
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def home():

    return {
        "message": "Agentic Text-to-SQL API"
    }


@app.get("/health")
def health():

    return {
        "status": "healthy"
    }
```

Run:

```bash
```

```
uvicorn app.main:app --reload
```

Open:

```
http://127.0.0.1:8000
```

You should see:

```
JSON

{
    "message": "Agentic Text-to-SQL API"
}
```

Open the automatically generated API documentation:

```
http://127.0.0.1:8000/docs
```

Step 14: Create the HTML frontend

Create:

```
frontend/index.html
```

Add:

```
HTML

<!DOCTYPE html>
<html>

<head>
    <title>Agentic Text-to-SQL</title>
</head>

<body>

    <h1>Agentic Text-to-SQL</h1>

    <input
        id="question"
        type="text"
        placeholder="Ask a question">

    <button onclick="submitQuestion()">
        Submit
```

Open `index.html` in your browser.

Step 15: Connect the HTML page to FastAPI

Modify `app/main.py`:

Python Run

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)


class Query(BaseModel):

    question: str


@app.post("/query")
def query(payload: Query):

    return {
        "question": payload.question,
        "status": "received"
    }
```

Modify `frontend/index.html`:

HTML

```html
<script>

async function submitQuestion() {

    const question =
        document.getElementById(
            "question"
        ).value;
```

```
    const response =
        await fetch(
            "http://127.0.0.1:8000/query",
            {
                method: "POST",
                headers: {
                    "Content-Type":
                        "application/json"
```

Restart FastAPI:

```bash
uvicorn app.main:app --reload
```

Test it.

Type:

```
Show the top 5 artists by sales
```

Expected result:

```json
{
    "question": "Show the top 5 artists by sales",
    "status": "received"
}
```

Phase 1 milestone

```
User
 ↓
HTML
 ↓
FastAPI
 ↓
Python
 ↙      ↘
SQLite  FalkorDB

Ollama ✓
```

At this point, we'll have all the infrastructure ready.

My recommendation is to stop here and verify that all four components work:

- SQLite
- FalkorDB
- FastAPI
- HTML → FastAPI communication

Then we'll start Phase 2, where we'll build the first real Text-to-SQL pipeline:

```
Question
 ↓
Ollama
 ↓
SQL generation
 ↓
SQLite execution
 ↓
Results
```

That phase will take only about **200-300 lines of code**, and you'll have your first working AI-powered Text-to-SQL application.